RELEASE ENGINEERING / DEVOPS

# Salesforce Release Management & CI/CD Pipeline

A repeatable, auditable Dev → QA → UAT → Production deployment framework built on GitHub Actions and Salesforce CLI.

**Document type:** Process & Technical Reference

**Prepared for:** Caelius Consulting

**Repository:** check2.0

**Version:** 1.0 | August 2026

CONTENTS

# Table of Contents

SECTION 01

# Executive Summary

This document defines the release management process and CI/CD pipeline that governs how Salesforce metadata moves through the organization's environment chain — from initial development to production — in a controlled, repeatable, and fully auditable manner.

The pipeline automates deployment mechanics while keeping **promotion decisions in human hands**: an engineer decides when code is ready to move from one environment to the next by merging it into the corresponding branch; the pipeline takes over from that point, validating, packaging, testing, and deploying exactly what changed — nothing more.

> **Core guarantee:** only the components introduced by the approved merge are identified and deployed to the target org. Components already present from a prior successful release are never redeployed or disturbed.

## Objectives Achieved

- **Complete four-stage flow implemented** — a single, version-controlled pipeline definition governs Dev, QA, UAT, and Production, each mapped to its own Salesforce org and GitHub Environment.
- **Change-scoped deployment** — a git-tag-based baseline mechanism (via `sfdx-git-delta`) ensures every deployment package reflects only the delta since that environment's last successful release.
- **Automated quality gates** — static analysis (ESLint) and full Apex test execution (`RunLocalTests`) run before any metadata reaches an org.
- **Human governance where it matters** — Production deployments pause for a mandatory reviewer approval before execution.
- **End-to-end demonstration completed** — a sample Apex change was progressed live through Dev → QA → UAT → Production, validating every stage of the design under real conditions.

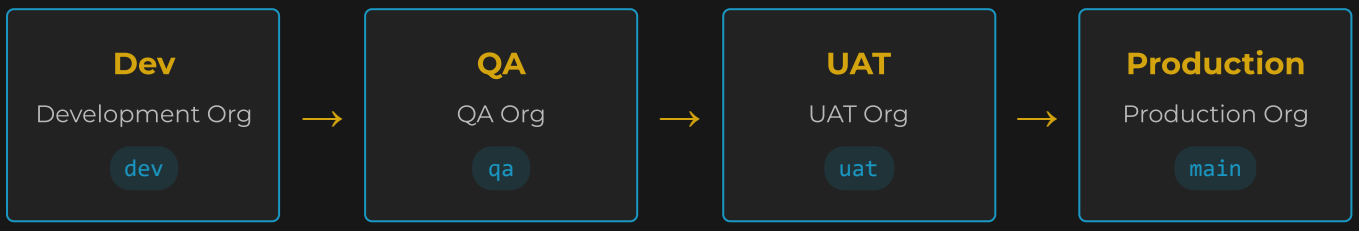
## Who This Document Is For

Sections 1–3 and the closing roadmap are written for stakeholders who need to understand *how releases move and are controlled*, without requiring CI/CD or Salesforce DX background. Sections 4 onward provide the technical detail engineers need to operate, extend, or audit the pipeline.

SECTION 02

# Solution Architecture

The architecture maps four Git branches to four Salesforce orgs, one per environment, through four matching GitHub Environments. This 1:1:1 relationship is what allows a single workflow definition to safely serve every stage of the release chain.




Promotion between stages is a deliberate merge action; deployment follows automatically once merged.

## Architectural Principles

| PRINCIPLE | IMPLEMENTATION |
|---|---|
| **Single source of truth** | One workflow file ( `deploy.yaml` ) governs all four environments — no per-environment duplication to keep in sync. |
| **Environment parity** | Every environment runs the identical validation → package → test → deploy sequence; only the target org and its gating rules differ. |
| **Least privilege** | The workflow is read-only by default; write access is elevated only for the single step that records a successful release. |
| **Segregation of credentials** | Each org's connection credential is scoped to its own GitHub Environment and is never visible to, or usable from, another environment's job. |
| **Progressive governance** | Control tightens closer to Production — Dev/QA/UAT deploy straight through on a passing pipeline; Production requires explicit human approval. |

SECTION 03

# Trigger & Promotion Model

Promotion between environments is intentionally **manual and deliberate** — a release owner decides when work is ready to move forward. Deployment execution is **automatic** the moment that decision is made. This separation gives teams full control over pacing (code may sit in QA for validation over multiple days or weeks) without sacrificing consistency or requiring anyone to remember to trigger a deployment by hand.

## How a Promotion Happens

1. An engineer merges the source branch into the target branch (e.g. `dev` into `qa`) — either directly, or via a reviewed pull request.
2. The merge lands on GitHub as a push to the target branch.
3. That push automatically starts the pipeline, scoped to the environment owned by that branch.

## Trigger Mechanisms

| TRIGGER | PURPOSE |
|---|---|
| Push | Primary trigger. Any push to `dev`, `qa`, `uat`, or `main` deploys to that branch's org. This is what makes an approved merge automatically initiate deployment. |
| Manual Dispatch | Secondary trigger, used to re-run a deployment without a new commit (e.g. retrying after a transient infrastructure failure). Requires selecting a target environment from a fixed list. |

> **Built-in safeguard:** when a deployment is manually dispatched, the pipeline verifies that the environment selected actually belongs to the branch the run was started from. A run started from `dev` cannot be pointed at Production — the pipeline rejects the mismatch before anything executes.

SECTION 04

# Pipeline Stages

Every run executes the same four stages, in the same order, regardless of which environment it targets.

## Stage 1 — Validate Deployment

Determines which environment and org this run targets by resolving the triggering branch (or, for a manual run, the selected input) into an environment name and Salesforce org alias. This resolved identity is passed to every later stage, so the rest of the pipeline never has to re-derive it.

## Stage 2 — Lint

Runs static analysis over Lightning Web Component and Aura JavaScript using Salesforce's recommended rule sets, catching platform-specific issues (Locker Service violations, invalid wire adapter usage, accessibility gaps) before code ever reaches an org. Currently configured as advisory while the codebase's UI layer is being built out; promotable to a hard gate with a single configuration change.

## Stage 3 — Generate Delta Manifest

The core of the "only what changed deploys" guarantee. Establishes a baseline — the last commit successfully deployed to this specific org — and computes everything that has changed since. Produces a deployment manifest containing exactly those components, plus a destructive-changes manifest if anything was removed. Full detail in Section 5.

## Stage 4 — Deploy to Salesforce

Authenticates to the target org using that environment's dedicated credential, validates the manifest against the org with a full test run, and — only if that validation passes — commits the change. On success, marks this org's new baseline for the next release cycle. For Production, this stage pauses for mandatory human approval before executing.

**Stage dependency chain:** Deploy only executes once Validate, Lint, and Generate Delta Manifest have all completed successfully. A failure at any earlier stage stops the release before it can reach an org.

SECTION 04 (continued)

## The Validate-Then-Deploy Pattern

The Deploy stage does not simply push metadata to an org and hope for the best. It follows a two-step pattern designed to make failure impossible to miss and success unambiguous:

### 1. Validation (Dry Run)

The full manifest is checked against the target org, and every local Apex test is executed — but nothing is committed. This confirms the release will succeed, including meeting Salesforce's mandatory org-wide test coverage threshold, before the org is touched at all.

### 2. Quick Deploy

Only if validation passes, the exact already-validated result is committed to the org. Tests are not re-run — this step deploys precisely what was just proven safe, quickly and deterministically.

> **Why this matters:** because the deployed result is byte-identical to what was validated a moment earlier, there is no window in which "what was tested" and "what was deployed" can drift apart.

### Destructive Changes

When a release removes metadata (e.g. deletes a field or class), the pipeline automatically detects this and applies removals *after* the new and modified components are deployed — avoiding any window in which something still in use has already been removed.

SECTION 05

# Delta-Based Deployment Strategy

This is the mechanism that satisfies the requirement that **only components included in the approved release are deployed**, and that **previously deployed components are never unintentionally redeployed**.

## The Baseline Marker

Each org has a marker — a Git tag such as `deployed-qa` — that points at the exact commit last successfully deployed to it. This marker moves *only* when a deployment to that org succeeds. It deliberately does not move on failure, and it is not based on "the previous commit" or "yesterday's push," which is what makes it reliable across merges, retries, and long gaps between promotions.

## Two Operating Modes

| MODE | WHEN IT APPLIES | BEHAVIOR |
|---|---|---|
| Delta | The org already has a baseline marker from a prior successful release. | Computes the precise difference between the baseline and the new commit. Only changed or new components are packaged. |
| Full (Bootstrap) | The org has never had a successful release recorded. | Packages everything currently in the codebase, establishing the org's first baseline. Used exactly once per org. |

> **Production exception:** Production never silently falls back to a full deployment. If no baseline marker exists for Production, the pipeline stops and requires the baseline to be deliberately established first — preventing an unbaselined, uncontrolled full deployment from ever reaching live users.

## Validated in Practice

This mechanism was proven under real conditions: promoting a single new Apex class through the pipeline resulted in a deployment package containing *only that class* — components already present in the target org from an earlier release were correctly excluded from every subsequent deployment package.

SECTION 06

# Environments, Secrets & Access Control

Each of the four environments is configured as its own isolated GitHub Environment, giving the pipeline a clean way to keep credentials, approval rules, and deployment history fully segregated per org.

| ENVIRONMENT | BRANCH | APPROVAL REQUIRED | CREDENTIAL SCOPE |
|---|---|---|---|
| Development | `dev` | — | Dev org only |
| QA | `qa` | — | QA org only |
| UAT | `uat` | — | UAT org only |
| Production | `main` | ● Required reviewer | Production org only |

## Credential Design

Every environment stores its org connection under an identically named secret. Because the pipeline resolves its target environment dynamically at run time, the platform itself supplies the correct value automatically — there is no branching logic in the pipeline that decides which credential to use, which removes an entire class of possible misconfiguration.

## Permission Model

- **Default: read-only.** The pipeline cannot write to the repository unless explicitly elevated.
- **Single elevation point.** Write access is granted only to the step that records a successful deployment — and only for that step.
- **No cross-environment access.** A job deploying to QA has no visibility into UAT or Production credentials at any point.

SECTION 07

## Quality & Validation Gates

Three independent gates protect every environment before code is allowed to run in it:

### Gate 1 — Static Code Analysis

Lightning Web Component and Aura JavaScript is checked against Salesforce's platform-specific linting rules, catching runtime and security-relevant issues (e.g. Locker Service violations) before deployment.

### Gate 2 — Automated Apex Testing & Coverage

Every deployment runs the org's full local Apex test suite as part of validation. Salesforce enforces a minimum 75% org-wide code coverage threshold on any deployment that includes Apex — the pipeline surfaces this as a clear, actionable failure rather than a silent rejection, including the exact coverage percentage and which tests failed, if any.

### Gate 3 — Human Approval (Production)

Once a release passes every automated gate, Production deployment still pauses for an explicit, named reviewer to approve it before execution proceeds. This is the final control point before a change reaches live users.

**Proven under real failure conditions:** during pipeline validation, a component with insufficient test coverage was correctly rejected at the automated testing gate, and the pipeline was hardened to surface the exact Salesforce error (coverage percentage, failing component) directly in the run log rather than a generic failure message — ensuring failures are always diagnosable without manual investigation.

SECTION 08

## Auditability & Release Tracking

Every deployment produces a traceable record without requiring any additional tooling:

| RECORD | WHAT IT SHOWS |
|---|---|
| **Pipeline run history** | A complete, timestamped log of every deployment attempt per environment, with pass/fail status for each stage. |
| **Release summary** | A human-readable summary published with every run, showing the exact components included and, if applicable, the components removed. |
| **Baseline markers** | A persistent, per-environment pointer to the last successfully deployed commit — a running record of "what is actually live" in each org. |
| **Environment deployment log** | A native, chronological history of every deployment to each environment, including who approved Production releases and when. |

> Because release status is derived from these records rather than tracked manually, "is this change in UAT yet?" is always answerable directly from the pipeline — without asking anyone.

SECTION 09

# Failure Handling & Containment

A failure at any stage is designed to fail loudly and stop cleanly, never to deploy a partial or unvalidated result.

## What Happens When a Stage Fails

- **Validation failure** (bad branch/environment combination): the run stops immediately, before any org is contacted.
- **Test/coverage failure**: the dry-run step fails, the deployment is never committed, and the org's baseline marker does not move — the next attempt will still include everything from this failed attempt.
- **Infrastructure failure** (credential missing, org unreachable): a clear, specific error identifies exactly what is missing and where to fix it, rather than a generic failure.

## Containment by Design

- **No partial deployments.** The validate-then-deploy pattern (Section 4) guarantees a failed validation never results in a partial commit to the org.
- **No silent progression.** Because promotion between environments is a deliberate merge action, a failed release cannot accidentally "carry forward" into the next environment on its own.
- **Full diagnosability.** Every failure surfaces the underlying platform error directly in the run log.

SECTION 10

## Governance & Roadmap

The current design meets the release management objective today. The following enhancements are recommended as the team and codebase mature, and require configuration changes only — no changes to the pipeline's core logic.

| ENHANCEMENT | VALUE |
|---|---|
| **Required pull-request reviews on QA / UAT / Production branches** | Formalizes "approved merge" as a reviewed, recorded decision rather than a direct push, once a second reviewer is available. |
| **Required status checks before merge** | Mechanically prevents promoting a commit whose deployment to the prior environment failed — closing the one gap between "pipeline enforces quality" and "process enforces quality." |
| **Promote linting from advisory to blocking** | Once Lightning Web Components and Aura components exist in the codebase, code-quality issues will actively block deployment rather than only being reported. |
| **Extend approval gating to QA/UAT** | Optional additional checkpoint if any environment beyond Production needs a manual sign-off before deployment executes. |
| **Periodic credential verification** | Scheduled confirmation that each environment's org connection remains valid, surfacing revoked or expired credentials before a release depends on them. |

SECTION 11

# Operational Runbook

## Promoting a Release to the Next Environment

```
git fetch origin
git checkout <target-branch>        # e.g. qa
git merge origin/<source-branch>    # e.g. dev
git push origin <target-branch>     # this push triggers deployment
```

Alternatively, open a pull request from the source branch into the target branch and merge it once reviewed — the resulting push triggers deployment identically.

## Establishing Production's First Baseline

Before the very first Production release, the baseline marker must be created manually, at the commit that reflects Production's current actual state:

```
git tag deployed-prod <commit-sha>
git push origin deployed-prod
```

## Re-running a Deployment Without a New Commit

From the Actions tab, run the workflow manually and select the target environment. The environment chosen must match the branch the run is started from.

## Responding to a Failed Deployment

1. Open the failed run and locate the failing stage.
2. Review the surfaced error — component failures, test failures, or coverage shortfalls are printed directly in the log.
3. Fix the underlying issue in the source branch (not the target branch) and re-promote.

SECTION 11 (continued)

## End-to-End Demonstration Record

The complete release flow was executed against real Salesforce orgs to validate the design, not merely to review it on paper. Summary of the demonstration run:

| STAGE | ACTION | RESULT |
|---|---|---|
| Dev | New Apex class merged and pushed to `dev` | 🟢 Deployed successfully; org-wide test coverage validated |
| QA | Promoted via merge to `qa` | 🟢 Delta-only package deployed; only the new component included |
| UAT | Promoted via merge to `uat` | 🟢 Delta-only package deployed successfully |
| Production | Promoted via merge to `main` | 🟡 Paused for required reviewer approval, then 🟢 deployed successfully upon approval |

> A second scenario was also exercised deliberately: a component was introduced with insufficient test coverage. The pipeline correctly rejected it at the validation gate, blocked the deployment, and left the environment's baseline unchanged. Once corrected and re-promoted, the same release deployed successfully — confirming failure containment works as designed, not just in theory.

SECTION 12

# Appendix — Reference & Glossary

## Environment Reference

| BRANCH | ENVIRONMENT | ORG ROLE | BASELINE MARKER |
|---|---|---|---|
| `dev` | Development | Developer Edition org | `deployed-dev` |
| `qa` | QA | Developer Edition org | `deployed-qa` |
| `uat` | UAT | Developer Edition org | `deployed-uat` |
| `main` | Production | Production org | `deployed-prod` |

## Glossary

| TERM | DEFINITION |
|---|---|
| **Delta** | The set of metadata components that differ between two points in the codebase's history — used to deploy only what changed. |
| **Dry Run** | A validation-only deployment that checks and tests a release against an org without committing any changes. |
| **Quick Deploy** | Committing a previously validated deployment result without re-running its tests. |
| **Baseline Marker** | A persistent pointer to the last commit successfully deployed to a given environment. |
| **Destructive Changes** | Metadata removals (deleted fields, classes, etc.) included in a release, deployed after additive changes. |
| **GitHub Environment** | A named deployment target within a repository that holds its own secrets, approval rules, and deployment history. |
| **Org-wide Test Coverage** | Salesforce's platform-enforced minimum (75%) of executable Apex code that must be covered by tests for a deployment containing Apex to succeed. |